

Project Design Phase

Solution Architecture — ACL Access Control

Date	2 October 2026
Team ID	
Project Name	Script-Controlled ACL — Branch-Based Record Access
Maximum Marks	4 Marks

Solution Architecture: Script-Controlled Record ACLs

The architecture places ServiceNow record ACL evaluation between user-facing interfaces and the Institution Details table. Each table operation is checked against its configured role; READ also uses the Branch = EEE data condition and the administrator exception described in the lab.

- Enforce access at the record-ACL layer for each requested table operation.
- Separate the branch-scoped READ condition from the CREATE, WRITE, and DELETE role gates.
- Represent the user roles and branch values explicitly so allowed and denied outcomes can be tested.
- Validate requests through ServiceNow lists/forms and, where enabled, Service Portal; the source does not document a production deployment.

ACL architecture and access flow:

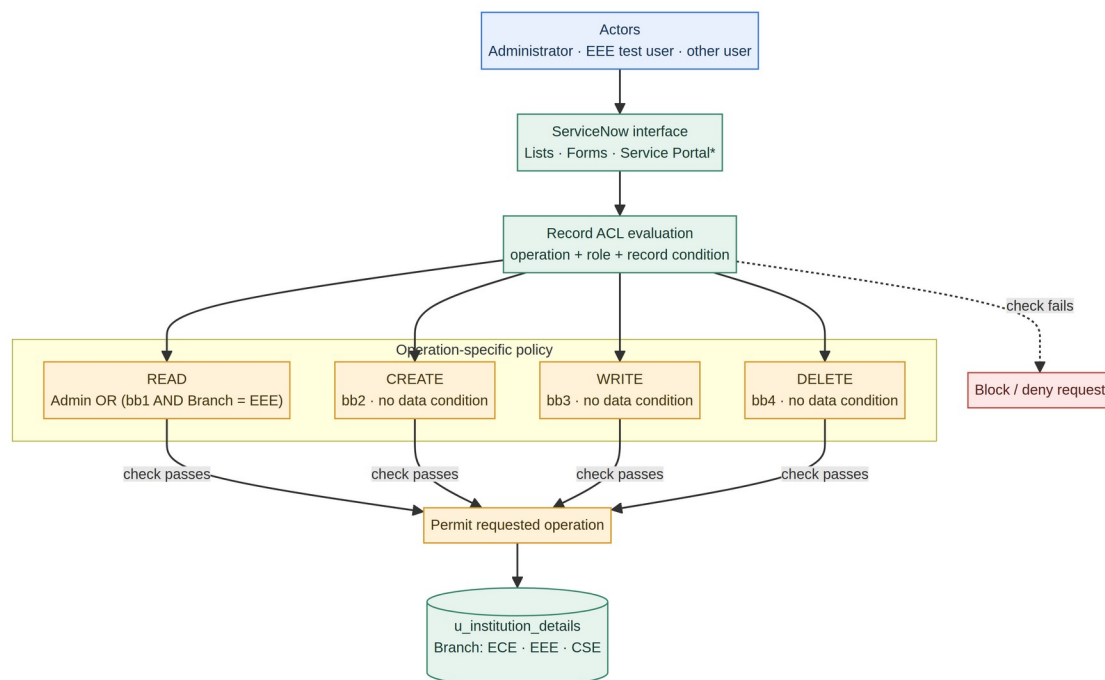


Figure 1. ServiceNow ACL flow and operation-specific role gates (based on the supplied lab).

Component	Responsibility / configuration
Actors and roles	Administrator and EEE test user; custom roles bb1 (READ), bb2 (CREATE), bb3 (WRITE), and bb4 (DELETE). The source says all four roles are assigned to its EEE User.
ServiceNow interface	Lists and forms are used in the lab; Service Portal is named in the outcome. Every enabled entry point should be checked against the same ACL policy.
Record ACL engine	Evaluates the operation-specific ACL. The READ ACL is advanced, active, and uses both a required role/script and a separate Branch = EEE data condition.
Protected table	Label: Institution Details. Name: u_institution_details. Seed records use Branch values ECE, EEE, and CSE.
Decision result	A permitted operation proceeds to the record; a failed authorization check denies the request. The source describes viewing/creating/editing/deleting verification, not logging or monitoring components.

Operation-level ACL matrix

Operation	ACL rule from lab	Verification interpretation
READ	Admin exception; otherwise bb1 script check plus data condition Branch = EEE.	EEE user sees EEE rows; non-bb1 sees none; admin sees all branches (per lab description).
CREATE	Required role bb2; no data condition specified.	Source checks New/create with the stated role combination; validate positive and negative role cases.
WRITE	Required role bb3; no data condition specified.	Source checks editing with bb1+bb2+bb3; branch-only write filtering is not specified by this ACL.
DELETE	Required role bb4; no data condition specified.	Source checks deletion with bb1+bb2+bb3+bb4; branch-only delete filtering is not specified by this ACL.

Table schema and verification notes

Field	Type / values
Student Roll Number	Auto Number
Student Name	Reference — User
Faculty Name	Reference — User
Branch	Choice — ECE, EEE, CSE
Email	String
Phone Number	String
Description	Multi String

Implementation and test notes — Role setup: the source assigns bb1–bb4 to EEE User but tests cumulative role sets; use separate test accounts or vary roles to isolate permissions. READ: test ECE/EEE/CSE; Branch = EEE is a separate condition for non-admin reads. CREATE/WRITE/DELETE: no data condition is specified, so do not infer EEE-only limits. Admin access is stated and explicit for READ; verify admin behavior for other operations.